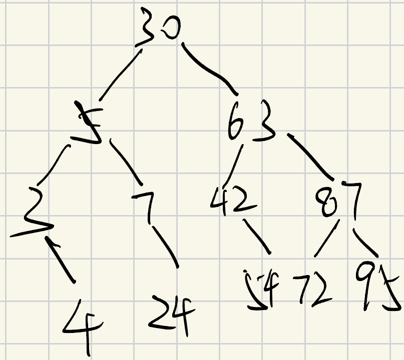
第七章

1. (1) C (2) D (3) C (4) A (5) B (6) D (7) C (8) C (9) C

(10) C (11) B (12) C (13) A (14) D (15) A

2. (1) ①

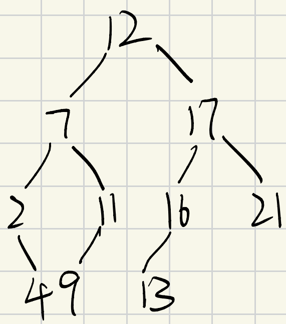


② 30、63、42、54

③ 30、63、87、95

④ $ASL = \dfrac{1+2\times2+4\times3+5\times4}{12} \approx 3.08$

(2)



(5) ①

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
关键字	32	17	16	63	49				24	40	10				30	31	46	47

② $15 = 63 \bmod 16$

故依次与31、46、47、17、16、63进行比较

③ $12 = 60 \bmod 16$

12号单位为空.查找失败

④ $$ASL = \frac{1\times6 + 2\times1 + 3\times3 + 6}{11} \approx 2.09$$

(6)

散列地址	0	1	2	3	4	5	6	7	8	9
关键字	14	1	9	23	84	27	55	20		
比较次数	1	1	1	2	3	3	1	2		

$$ASL = \frac{1\times4 + 2\times2 + 3\times2}{8} = 1.75$$

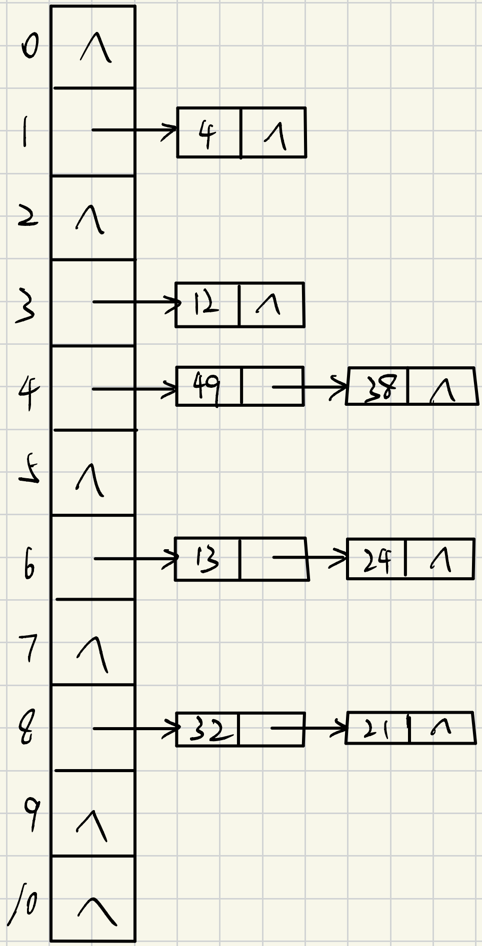
(7) ①

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字		4		12	49	38	13	24	32	21	
比较次数		1		1	1	2	1	2	1	2	

$$ASL_{succ} = \frac{1\times5 + 2\times3}{8} = 1.375$$

$$ASL_{unsucc} = \frac{1+2+1+8+7+6+5+4+3+2+1}{11} \approx 3.64$$

②



$$ASL_{succ} = \frac{1+1+1+2+1+2+1+2}{8} = \frac{11}{8}$$

$$ASL_{unsucc} = \frac{1+2+1+2+3+1+3+1+3+1+1}{11}$$

$$\approx \frac{19}{11}$$

3. (1)

```
int BinSearch(SSTable T, KeyType key, int low, int high){
    if (low > high){
        return -1;
    } else {
        int mid = (low + high) / 2;
        if (key = T.R[mid].key){
            return mid;
        } else if (key > T.R[mid].key){
            return BinSearch(T, key, mid+1, high);
        } else {
            return BinSearch(T, key, low, mid-1);
        }
    }
}
```

(2)
```
BiTree pre = NULL;
void JudgeBST(BTree T, int flag){
    if (T != null && flag){
        JudgeBST(T->lchild, flag);
        if (pre == null) {
            pre = T;
        } else if (pre->data < T->data){
            pre = T;
        } else {
            flag = 0;
        }
        JudgeBST(T->rchild, flag);
    }
}
```

```
(6) Status Insert_k(ElemType data){
        addr = H(data);
        temp = HT[addr];
        while (temp->next){
            if (temp->next->data == data){
                return ERROR;
            }else{
                temp = temp->next;
            }
        }
        s = new LNode;
        s->data = data;
        s->next = temp->next;
        temp->next = s
        return OK
}
```

```
Status Delete_K( ElemType data){
    addr = H(data);
    temp = HT[addr];
    while ( temp->next ){
        if (temp->next->data == data){
            S = temp->next;
            temp->next = S->next;
            delete S;
            return OK;
        }
        temp = temp->next;
    }
    return ERROR;
}
```